

Lab 4 Report – Performance Analysis A

In this report, I aim to analyze the performance of the game board display initialization and VGA interface interactions in the Game of Life application of Part 3. The function `GoL_draw_grid` is called to initialize the game board VGA display. It calls `VGA_draw_line` 13 times to draw the horizontal lines and 17 times to draw the vertical lines. `VGA_draw_line` calls `VGA_draw_point_ASM` once for each pixel on the VGA display in that line.

To record data about the number of instructions executed and the number of memory accesses, I created a main function that only calls `GoL_draw_grid`. Once compiled, I placed breakpoints at the beginning and end of all functions present: `GoL_draw_grid`, `VGA_draw_line` and `VGA_draw_point_ASM`. Then, using the Counters pane, we can determine the number of executed instructions, memory accesses and simulator time by subtracting the numbers at the start breakpoint from the end breakpoint. The results are shown in Table 1.

Table 1. Data of the performance analysis for functions in game board initialization.

	GoL_draw_grid	VGA_draw_line (horizontal line)	VGA_draw_line (vertical line)	VGA_draw_point_ASM
# of executed instructions at start	170	182	79760	203
# of executed instructions at end	149304	5959	84087	213
# of executed instructions	149134	5777	4327	10
# of data loads at start	58	58	156	59
# of data loads at end	268	59	157	59
# of data loads	210	1	1	0
# of data stores at start	47	53	8950	59
# of data stores at end	16742	699	9436	61
# of data stores	16695	646	486	2
Data / Instructions	11.34%	11.20%	11.25%	20%

From the results, we notice that in `GoL_draw_grid`, there are 149134 instructions executed in total, with $210 + 16695 = 16905$ data memory accesses. Thus, $16905/149134 = 11.34\%$ of the executed instructions were data memory accesses in the game board initialization. This means that if the time required to access memory were significantly longer than the emulator assumes, the overall runtime of the grid initialization would also increase substantially, since over $1/10^{\text{th}}$ of the instructions executed are data memory accesses.

In terms of time spent in each function, we can estimate it using the number of executed instructions, since the simulator runtime metric is unreliable as it depends on the emulator and varies every time the program is executed. `VGA_draw_line` is called a total of 30 times within `GoL_draw_grid`, making up $13 * 5777 + 17 * 4327 = 148660$ instructions executed. This represents $148660/149134 = 99.68\%$ of total executed instructions in the game board initialization. This is an overwhelming majority of time, with the only time spent not in `VGA_draw_line` being calling or returning from functions, and the looping for horizontal and vertical lines. Within `VGA_draw_line`, `VGA_draw_point_ASM` is called 320 times since there are 320 pixels horizontally on the VGA display. Thus, it makes up $320 * 10 = 3200$ instructions spent inside `VGA_draw_point_ASM` for a single horizontal line, which is $3200/5777 = 55\%$ of the time of `VGA_draw_line`. The rest of the time is spent calling and returning from functions, doing input validation and executing control flow (to determine if the given points are horizontal or vertical). Therefore, in the game grid initialization, we can estimate that around half of the time is spent executing instructions within `VGA_draw_line` (excluding function calls to `VGA_draw_point_ASM`), and the other half within `VGA_draw_point_ASM`.